

Data Structure

Level 1

Big $O(1)$: Constant Time
Function

Data Structure

Level 1

Complexity of an Algorithm

Big O

$O(1)$

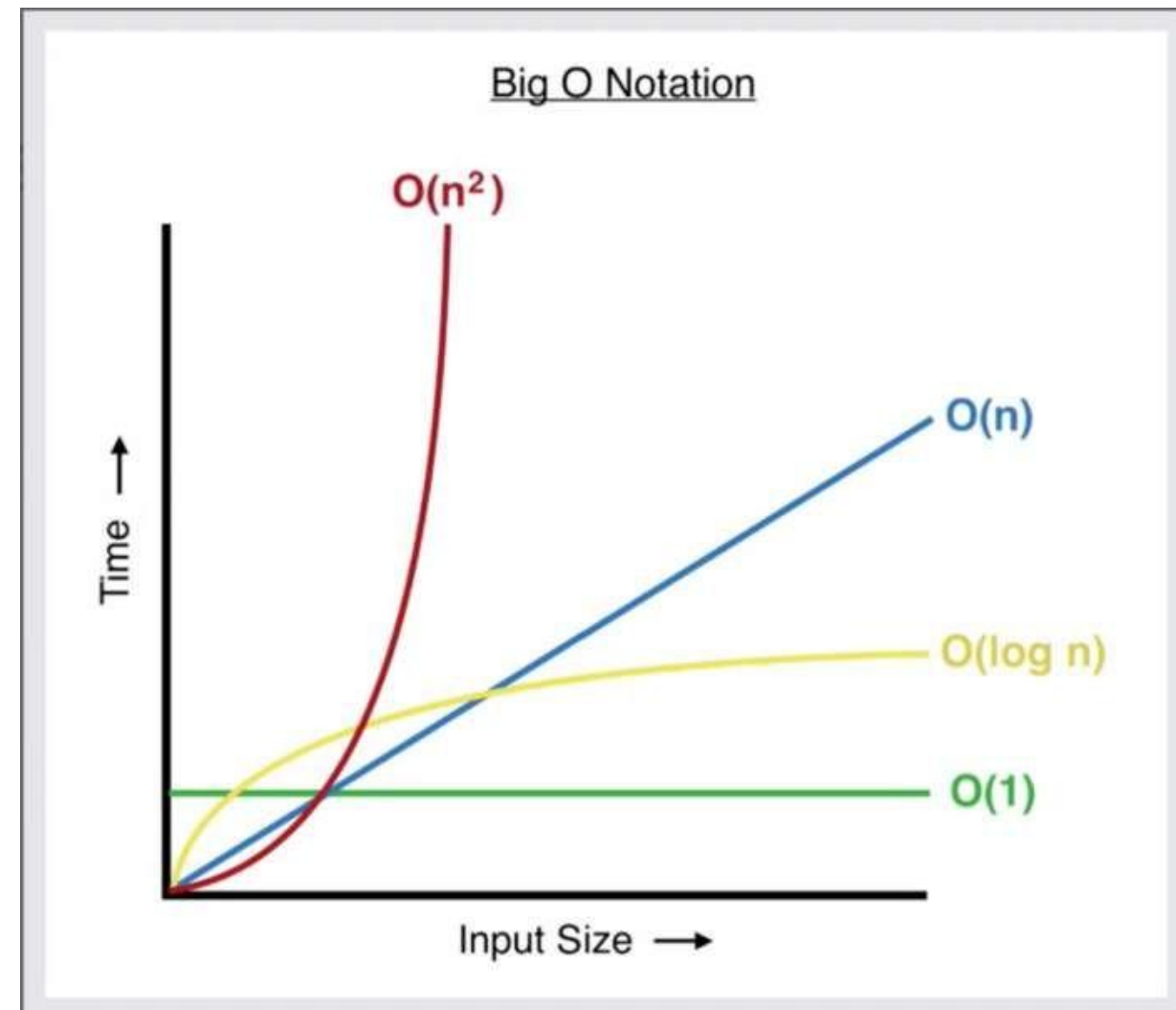
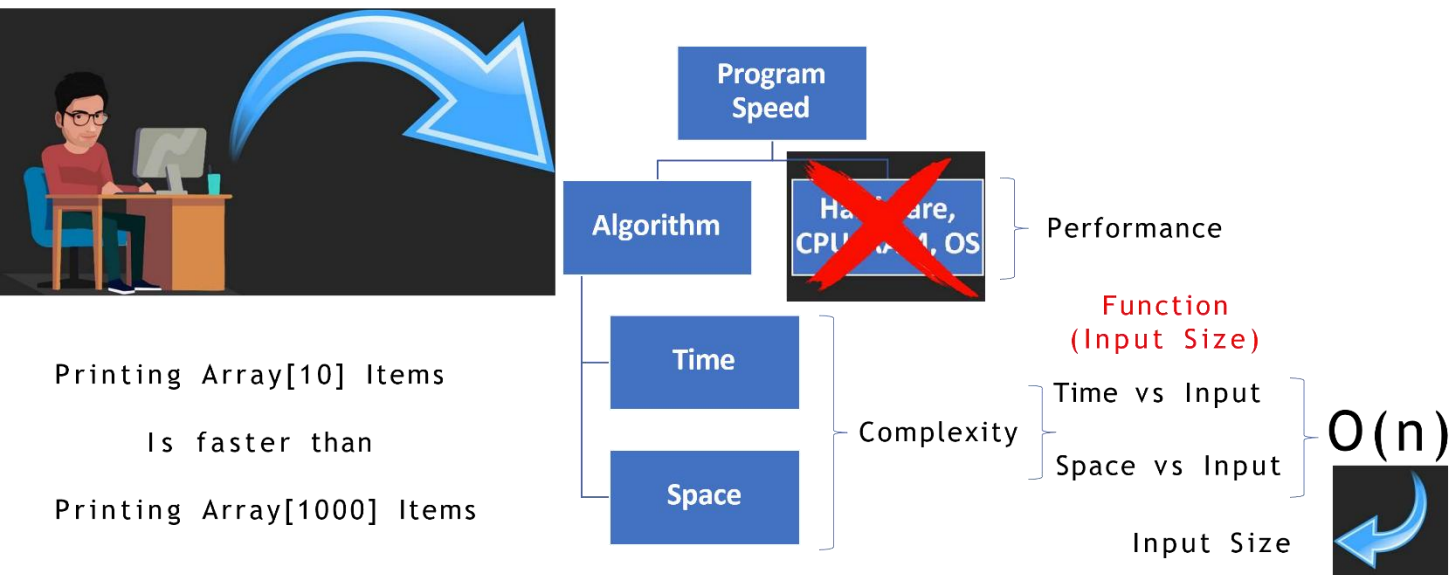
The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, the silhouettes of several people are visible, appearing to be in a collaborative setting, possibly a meeting or a workshop, with some individuals leaning over a table or screen.

Data Structure

Level 1

Quick Review

What is Big O?



Big O for the following Algorithms:

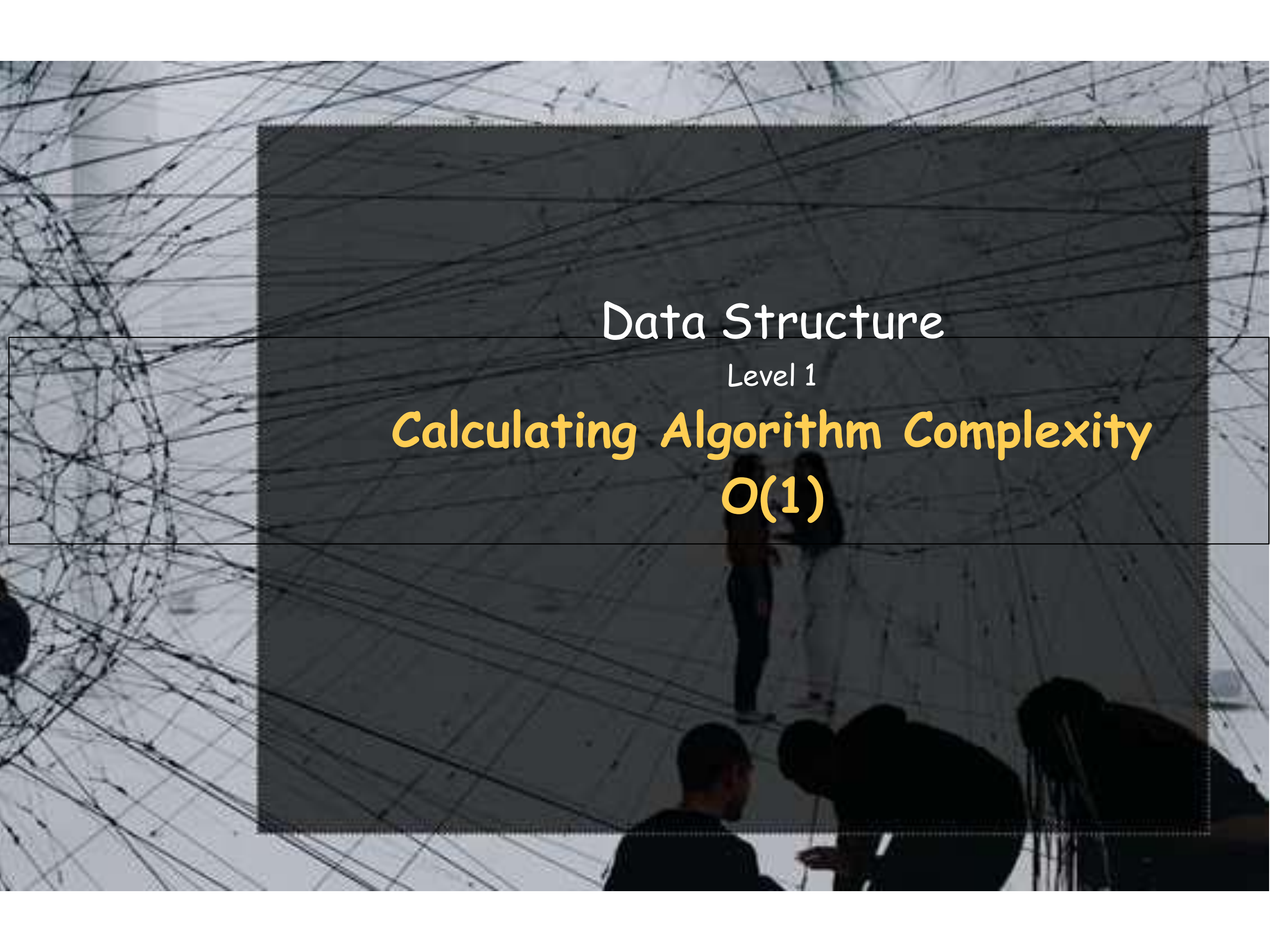
Algorithm 1

```
char GetLastCharacter(string S1)
{
    return S1[ S1.length() - 1 ];
}
```

Algorithm 2

```
char GetLastCharacter2(string S1)
{
    int n = S1.length() - 1 ;

    for ( int i = 0 ; i <= n ; i++ )
    {
        if ( i == n )
        {
            return S1 [ n ] ;
        }
    }
}
```



Data Structure

Level 1

Calculating Algorithm Complexity $O(1)$

Calculating Algorithm Complexity:

Count the number of steps during execution.

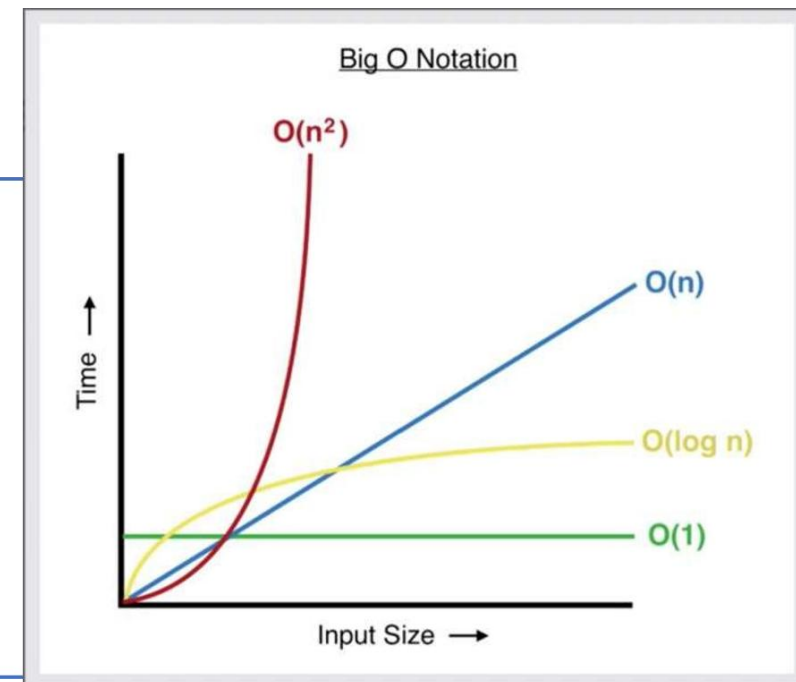
Simplification:

- Each step costs the same time.
- +, -, return, access array element, ... etc.

```
char GetLastCharacter(string S1)
```

```
{  
    return S1[S1.length() - 1];  
}
```

Big O = $4 * O(1) = 4 O(1)$
Remove Factor
 $O(1)$



- Number of Steps: 4 Steps, and it will be always 4 steps, it has nothing to do with array size
- Constant Time , independent of array size

Thank you 😊